THERS

Dialogo de exposicion: el Backend

Guion hablado, en orden, con puntos de referencia para mostrar cada cosa en el repositorio.

Como usar este documento. Cada seccion tiene un texto para decir en voz alta (sin leer codigo) y un recuadro azul con *rutas exactas* para abrir en el editor o en la documentacion si alguien pregunta o si quieres demostrar algo. Las rutas son relativas a la raiz del repositorio `THERS_REDSOCIAL/`. Tiempo estimado hablando: 12 a 15 minutos.

Cifras de apoyo (contadas en el repositorio): ~15 000 lineas de Python en `backend/app`; 38 ADR (decisiones de arquitectura) en `docs/architecture/`; 34 migraciones en `backend/migrations/versions/`; 36 archivos de prueba en `backend/tests/`; unos 80 endpoints en las rutas (`CLAUDE.md` registra 70 combinaciones documentadas antes de moderacion y reportes; si citas un numero exacto, verificalo en `API_CONTRACT.md`); 592 pruebas de integracion segun la ultima verificacion registrada en `CLAUDE.md` (no las volvi a ejecutar para preparar este guion).

Indice: 1 Vision general - 2 Tecnologias - 3 Arquitectura en capas - 4 Una peticion de punta a punta - 5 Base de datos - 6 Autenticacion y registro - 7 Sesiones y tokens - 8 Seguridad - 9 Funcionalidades sociales - 10 Privacidad - 11 Moderacion - 12 Calidad - 13 Estado y cierre - Anexo: preguntas probables.

1. Vision general

Buenos dias. Voy a presentar el backend de THERS, que es el cerebro de nuestra red social: todo lo que el usuario no ve, pero que hace que la aplicacion funcione. Es la API que consumen tanto la pagina web como la aplicacion movil Android.

Para ubicarnos: el repositorio es un monorepo con cuatro aplicaciones independientes: **backend**, el **Frontend** web, el **handbook** de ingenieria y la app **mobile**. Hoy me centro solo en el backend, hecho en Python con Flask y con PostgreSQL como base de datos.

No es un prototipo de login: cubre registro con verificacion por correo, inicio de sesion con Google, doble factor, publicaciones, likes, comentarios, seguidores, mensajes directos, notificaciones, privacidad, moderacion y exportacion de datos.

> **PUNTOS DE REFERENCIA - donde encontrarlo**
> • `backend/app/` - codigo; punto de entrada: `backend/run.py`
> • Indice de reglas y estado del proyecto: `CLAUDE.md` (raiz)
> • Arquitectura del backend: `docs/architecture/BACKEND_ARCHITECTURE.md`

2. Tecnologias

Elegimos **Flask** por ser ligero y explicito: no impone una estructura, asi que pudimos disenarla nosotros. Para la base de datos usamos **PostgreSQL** con **SQLAlchemy** como ORM y **Flask-Migrate** (Alembic) para versionar los cambios del esquema, como un git para la base de datos.

La autenticacion usa **JWT** con Flask-JWT-Extended. Para el doble factor usamos **pyotp** (TOTP, el estandar de Google Authenticator), para verificar Google usamos **google-auth**, para correos **Resend**, para procesar imagenes **Pillow** y para guardarlas **boto3** (compatible con S3, Supabase o Cloudflare R2). En produccion se sirve con **gunicorn**.

Un detalle de criterio: cada libreria externa esta confinada a un solo archivo. Si manana cambiamos de proveedor de correo, se toca un archivo, no veinte.

> **PUNTOS DE REFERENCIA - donde encontrarlo**
> • Dependencias fijadas con version: `backend/requirements.txt` (cada una con un comentario del por que)
> • Dependencias de pruebas: `backend/requirements-dev.txt`
> • Wrappers de librerias externas: `backend/app/infrastructure/` (`email/`, `auth/`, `media/`)
> • PostgreSQL local con Docker: `docker-compose.yml` (raiz); guia: `docs/LOCAL_DEV_SETUP.md`

3. Arquitectura en capas

La decision de diseno mas importante es que el backend esta organizado en **capas**, inspiradas en Clean Architecture. Piensen en un restaurante: el mesero toma el pedido, el chef cocina siguiendo las reglas de la casa y el almacen guarda los ingredientes. Cada uno hace una sola cosa.

Interfaces es el mesero: recibe la peticion HTTP y devuelve la respuesta. **Application** son los casos de uso: una carpeta por funcionalidad, y cada archivo hace una accion, por ejemplo 'crear

post' o 'iniciar sesion'. **Domain** son las reglas del negocio puras, como 'un post tiene maximo 2000 caracteres' o 'hay que tener 18 anos'; no sabe que existe Flask ni PostgreSQL. Y **Infrastructure** es el almacen: ahi viven SQLAlchemy, el envio de correos y el guardado de imagenes.

La regla clave es que las dependencias apuntan hacia adentro. Domain define un 'contrato' (un repositorio abstracto) e Infrastructure lo implementa. Esto permite probar la logica sin depender de una tecnologia concreta y cambiarlas sin reescribir las reglas.

PUNTOS DE REFERENCIA - donde encontrarlo

- Capas: `backend/app/interfaces/`, `application/`, `domain/`, `infrastructure/`
- Contrato abstracto: `app/domain/posts/repositories.py` - implementacion real: `app/infrastructure/persistence/repositories/post_repository.py`
- Regla pura: `app/domain/posts/validators.py` (maximo 2000 caracteres)
- Explicacion formal: `BACKEND_ARCHITECTURE.md`, secciones de capas y flujo de una peticion

4. Una peticion de punta a punta

Sigamos un ejemplo: un usuario publica un post. La peticion `POST /api/posts` llega a la capa de interfaces, a `post_routes.py`. Ahi se comprueba el token JWT y se lee el JSON. Luego se llama al caso de uso `create_post_use_case`, que valida el texto con las reglas del dominio, pide al repositorio que guarde el post y procesa las menciones con @ para crear notificaciones. Por ultimo, un 'presenter' convierte el resultado a la forma publica del JSON, sin exponer datos internos.

Todo error sale con el mismo formato, `{"msg": "..."}`, incluso los inesperados: hay un manejador global, asi que el cliente nunca recibe una pagina HTML de error. Ademas, toda respuesta a una peticion autenticada lleva `Cache-Control: no-store`, un hallazgo de la prueba en un telefono real, donde la capa de red guardaba en cache datos del usuario.

PUNTOS DE REFERENCIA - donde encontrarlo

- Ruta: `app/interfaces/routes/post_routes.py` - caso de uso: `app/application/posts/create_post_use_case.py` - presenter: `app/application/posts/post_presenter.py`
- Fabrica de la app y registro de todos los blueprints: `app/__init__.py` (`create_app()`)
- Manejador global de errores: `app/interfaces/error_handlers.py`
- Formato de request/response/error: `docs/architecture/API_CONTRACT.md` (seccion 3)

5. Base de datos

El modelo de datos tiene unas 20 tablas. El nucleo es **users**, y alrededor giran **posts**, **likes**, **comments**, **follows**, **notifications** y **messages**. Hay tablas de seguridad: **sessions**, **refresh_tokens**, **two_factor_recovery_codes**, **password_reset_tokens** y **email_verification_tokens**. Y tablas de control y cumplimiento: **user_restrictions** (bloqueos), **muted_keywords**, **muted_topics**, **mentions**, **reports**, **data_exports**, **rate_limit_buckets** y **account_deletion_codes**.

Un punto de proceso del que estamos orgullosos: la base de datos nunca se modifica a mano. Cada cambio es una **migracion** versionada, y cada decision importante esta registrada en un **ADR**, un documento corto que explica el contexto, la decision y sus consecuencias. Son 38. El 'por que' de cada tabla esta escrito.

PUNTOS DE REFERENCIA - donde encontrarlo

- Todos los modelos (tablas): `app/infrastructure/persistence/models.py`
- Historial de migraciones: `backend/migrations/versions/`
- Diagrama y explicacion: `docs/architecture/DATABASE_ARCHITECTURE.md`, `DATABASE_ERD.md`
- Decisiones por tabla: `ADR-004` posts, `ADR-005` likes, `ADR-006` comentarios, `ADR-007` follows, `ADR-008` notificaciones, `ADR-013` mensajes

6. Autenticacion y registro

El registro es de varios pasos, y cada uno es deliberado. Primero se envia un **codigo de 6 digitos** al correo; la cuenta no puede iniciar sesion hasta verificarlo. Los codigos expiran a los 10 minutos, permiten maximo 5 intentos y hay 60 segundos de espera entre reenvios. Las contrasenas se guardan con **scrypt**, nunca en texto plano, y los propios codigos tambien se guardan hasheados.

THERS es solo para mayores de 18, y eso se valida en el servidor, no solo en la pantalla. Los nombres de usuario tienen su propia politica.

Tambien se puede entrar con **Google**: el servidor verifica criptograficamente el token que entrega Google; nunca confiamos en lo que diga el cliente. La recuperacion de contrasena sigue el mismo patron de codigos. Algo sutil: cuando el login falla, el mensaje es siempre el mismo, para no revelar si un correo existe o no.

PUNTOS DE REFERENCIA - donde encontrarlo

- Rutas de auth: `app/interfaces/routes/auth_routes.py`
- Casos de uso: `app/application/auth/` (`register_use_case.py`, `login_use_case.py`, `google_auth_use_case.py`, `forgot_password_use_case.py`)
- Reglas (18 anos, contrasena minima, tiempos de codigos): `app/domain/auth/validators.py` y `token_policy.py`
- Decisiones: `ADR-009`, `ADR-010`, `ADR-011` (verificacion obligatoria), `ADR-012` (Google), `ADR-034` (edad minima)

7. Sesiones, tokens y doble factor

Una vez dentro, el usuario recibe un **access token** JWT de 15 minutos y un **refresh token** de 30 dias. Cuando el primero expira, el cliente pide uno nuevo con el segundo, sin que el usuario escriba otra vez su contrasena. El refresh es **rotativo**: cada uso entrega uno nuevo e invalida el anterior, y si alguien intenta reutilizar uno ya gastado, se revoca toda la familia de tokens: asi un robo queda detectado y cortado. Esto esta verificado en un telefono real.

Ademas llevamos un **registro de sesiones**: cada token tiene una fila en la base de datos, de modo que el usuario puede ver sus dispositivos activos y cerrar cualquiera, e incluso un token firmado deja de servir si su sesion se cierra. Tambien hay alertas de inicio de sesion.

Para quien quiera mas proteccion existe **doble factor con TOTP** (app autenticadora) con codigos de recuperacion. Mientras se espera el segundo factor, el servidor emite un token de 'desafio' que a proposito no sirve para ningun otro endpoint.

PUNTOS DE REFERENCIA - donde encontrarlo

- Politica de tokens: `docs/architecture/ADR-017-jwt-session-policy.md`
- Hooks de JWT, revocacion y errores 401 uniformes: `app/extensions.py`
- Refresh y logout: `auth_routes.py` (`/refresh`, `/logout`); repositorio: `infrastructure/persistence/repositories/refresh_token_repository.py`
- Sesiones: `app/application/sessions/`, `ADR-025` - Doble factor: `app/application/two_factor/two_factor_use_case.py`, `ADR-026`

8. Seguridad transversal

Ademas, hay medidas que atraviesan todo el sistema. Un **rate limiter** propio, guardado en la base de datos, limita intentos: por ejemplo 10 intentos de login por 15 minutos, 5 intentos de segundo factor por 15 minutos, 5 registros por hora. Se limita por cuenta y tambien por IP, porque un atacante rota IPs facilmente pero ataca una cuenta concreta.

El **CORS** se restringe en produccion a los origenes web configurados. Los **secretos** vienen de variables de entorno, y si falta la clave JWT la aplicacion se niega a arrancar salvo que se active explicitamente un modo de desarrollo inseguro. Las imagenes subidas se validan por su contenido real y se re-codifican, lo que elimina metadatos EXIF como la ubicacion.

Para privacidad legal, cada usuario puede **exportar sus datos** y **eliminar su cuenta** con confirmacion por codigo, y una politica de retencion purga sesiones antiguas a los 90 dias.

PUNTOS DE REFERENCIA - donde encontrarlo

- Rate limiting: `app/domain/rate_limiting/policy.py` (todos los limites), `app/application/rate_limiting/rate_limit_guard.py`, `ADR-027`
- Configuracion y secretos: `app/config.py`, plantilla `backend/.env.example` (el .env real no se sube)
- Imagenes: `app/infrastructure/media/image_processor.py`, `ADR-015`
- Exportar datos: `app/application/data_exports/`, `ADR-028` - eliminar cuenta: `app/application/account_deletion/`, `ADR-031` - retencion: `ADR-037`

9. Funcionalidades sociales

Pasemos a lo que hace a THERS una red social. Se pueden **publicar posts** de hasta 2000 caracteres, **editarlos** y **borrarlos**; **dar like**; **comentar**; y **seguir** a otras personas, con sugerencias de a quien seguir. Las **menciones** con @ generan notificaciones automaticamente.

Hay **mensajes directos** con edicion, borrado e indicador de 'escribiendo...'. Con honestidad tecnica: hoy no usamos WebSockets; el cliente consulta periodicamente (*polling*) mensajes y notificaciones. Es una decision de simplicidad para esta etapa.

Las **notificaciones** se crean cuando alguien te sigue, te da like, te comenta o te menciona.

PUNTOS DE REFERENCIA - donde encontrarlo

• Posts: `app/interfaces/routes/post_routes.py`, `app/application/posts/` - ADR-004, ADR-019, ADR-021

• Likes: `like_routes.py`, ADR-005 - Comentarios: `comment_routes.py`, ADR-006, ADR-020

• Follows y solicitudes: `follow_routes.py`, `app/application/follows/`, ADR-007

• Mensajes: `message_routes.py`, `app/application/messages/`, ADR-013, ADR-014, ADR-035 - Menciones: `app/application/mentions/`, ADR-023

• Notificaciones: `notification_routes.py`, ADR-008

10. Privacidad y control del usuario

Una red social responsable le da el control al usuario. Se puede tener una **cuenta privada**: quien quiera seguirte debe ser aceptado, y tus posts solo los ven tus seguidores. Ese filtro se aplica en la consulta SQL, no despues en Python, para que las paginas del feed no queden incompletas.

Cada usuario puede **bloquear** y **restringir** cuentas, **silenciar palabras y temas** que no quiere ver, y elegir sus preferencias de privacidad y de contenido sensible.

PUNTOS DE REFERENCIA - donde encontrarlo

• Cuentas privadas: `ADR-022`, `app/application/privacy/post_visibility.py`

• Rutas: `app/interfaces/routes/privacy_routes.py`, `restriction_routes.py`

• Palabras y temas silenciados: `app/domain/moderation/`, `app/application/moderation/`, ADR-024

• Bloqueos y restricciones: `app/application/restrictions/`, ADR-029 - preferencias de contenido: ADR-030

11. Moderacion y seguridad infantil

Como la app tiene contenido generado por usuarios, Google Play exige poder **reportar** y bloquear contenido y personas, y exigir **aceptar terminos de uso**. Lo implementamos: se pueden reportar posts, comentarios, mensajes y usuarios con un motivo (spam, acoso, odio, violencia, etc.). Los moderadores, cuentas con un permiso especial, revisan una cola de reportes y resuelven con acciones.

Los reportes de explotacion infantil reciben siempre prioridad critica asignada por el servidor. Debo ser transparente: el ADR-032 de moderacion figura como **propuesto, pendiente de ratificacion** por el equipo.

PUNTOS DE REFERENCIA - donde encontrarlo

• Reportes: `app/interfaces/routes/report_routes.py`, `app/application/reports/report_use_cases.py`, motivos en `app/domain/reports/kinds.py`

• Cola de moderadores: `app/interfaces/routes/moderation_routes.py`, `app/application/platform_moderation/`, permiso en `app/interfaces/moderator_required.py`

• Comandos de consola (`flask set-moderator`, `unsuspend-user`): `app/interfaces/cli.py`

• Decisiones: ADR-032, ADR-038 - Terminos: `app/application/terms/`, `docs/legal/` - Requisitos de la tienda: `docs/LAUNCH_CHECKLIST.md`

12. Calidad: pruebas y documentacion

El backend tiene una suite de **pruebas de integracion** con pytest, y lo importante es que corren contra un **PostgreSQL real**, no contra simulaciones. Hay archivos de prueba por funcionalidad: autenticacion, posts, mensajes, rate limiting, moderacion y mas. La ultima verificacion registrada en la documentacion fue de 592 pruebas aprobadas.

Hay un **contrato de API** que cataloga cada endpoint; la regla es que todo endpoint nuevo se documenta el mismo dia. Y trabajamos con **Git Flow**: ramas por funcionalidad, *Conventional Commits* y Pull Requests con revision; el repositorio tiene mas de 220 commits.

PUNTOS DE REFERENCIA - donde encontrarlo

- Pruebas: `backend/tests/` (base de pruebas en `conftest.py`); ejecutar `python -m pytest` desde `backend/`
- Contrato de la API: `docs/architecture/API_CONTRACT.md`
- Gobernanza y Git Flow: `docs/architecture/organization/` (HB-001); plan de ramas: `docs/BRANCH_AND_PR_PLAN.md`
- Despliegue en staging: `docs/STAGING_DEPLOY_GUIDE.md`, `ADR-018`

13. Estado, limites y cierre

Para cerrar, con honestidad sobre el estado actual. Lo que esta **hecho y funcionando**: todo el backend descrito, consumido de punta a punta por la web y, en login, perfil y sesion, por la app movil, verificado en un telefono real.

Lo que **todavia no esta**: tiempo real con WebSockets (hoy hay polling); la web aun no usa el refresh token, por lo que su sesion dura 15 minutos; DevOps (CI/CD, dominios, monitoreo) no tiene documentacion oficial y el ADR-018 de hosting es una propuesta; y la ratificacion formal de los documentos de arquitectura por el Comite Tecnico esta pendiente.

En resumen: construimos un backend por capas, seguro por diseno, con cada decision documentada y probada. Muchas gracias. Con gusto respondo preguntas.

PUNTOS DE REFERENCIA - donde encontrarlo

- Estado y huecos reconocidos: `CLAUDE.md`, secciones 5 y 15
- Hoja de ruta y validacion movil: `docs/mobile/ROADMAP.md`, `docs/mobile/VALIDATION.md`
- Checklist de lanzamiento: `docs/LAUNCH_CHECKLIST.md`

Anexo: preguntas probables y respuestas cortas

P: Por que Flask y no Django?

R: Flask es minimo y nos dejo disenar nuestra propia estructura por capas. (Es una explicacion de criterio; no encontre un ADR de esta eleccion, no la presentes como decision registrada.)

P: Que pasa si roban un token?

R: El access dura 15 minutos y esta ligado a una sesion revocable. El refresh es rotativo: reutilizarlo revoca toda la familia. Ver ADR-017 y ADR-025.

P: Como protegen las contrasenas?

R: Con scrypt (hash lento con sal), nunca en texto plano. Los codigos OTP tambien se guardan hasheados.

P: Por que no hay chat en tiempo real?

R: Decision de simplicidad: polling HTTP por ahora. No hay Socket.IO ni WebSocket en el backend; seria una evolucion futura.

P: Como evitan fuerza bruta?

R: Rate limiting propio en base de datos, por cuenta y por IP, mas limite de intentos en cada codigo OTP. Ver domain/rate_limiting/policy.py.

P: Como se cambia la base de datos?

R: Solo con migraciones versionadas (Alembic) y un ADR cuando la decision es relevante.

P: Esta todo ratificado?

R: No: los documentos de arquitectura estan pendientes de ratificacion formal segun HB-001; lo decimos abiertamente.

P: Y los datos del usuario?

R: Puede exportarlos, eliminar su cuenta, ver y cerrar sesiones; las imagenes pierden metadatos EXIF.